음성 AI 전문가 과정 · Day 11 / 40

Day 11

한국어 음성 데이터셋
AIHub · KsponSpeech · Zeroth-Korean

데이터셋 지도 → HF datasets 스트리밍 → KsponSpeech 전사 규칙 · PCM → AIHub JSON → 목록표 → 품질 필터 → 화자 단위 분할 → Whisper 학습 입력
3 주차 M3 시작 · 4 시간 · 실습 노트북 Part 0 ~ 7

```
# ko_asr_manifest.csv
path,text,duration,speaker,source
zeroth_test/..flac,선생님께서 ..,4.1,z_105,zeroth
kspon_mini/..pcm,그럼 3 시에 ..,2.8,kspon_A,kspon
aihub_mini/..wav,오늘 회의는 ..,3.2,S001,aihub

→ 16 kHz · 필터 · 화자 단위 분할
→ input_features 80×3000 + labels
→ save_to_disk("ko_asr_prepared")
```

출력 모양 예시 · 값은 데이터에 따라 달라집니다

0.0 · 오늘 답할 질문

모델보다 먼저 데이터 : 무엇을 , 어떻게 깨끗하게 준비하나

1	한국어 음성 데이터셋은 무엇이 있고 , 무엇을 골라야 하나 ?	Part 1
2	Hugging Face 데이터셋을 디스크를 아끼며 어떻게 살펴보나 ?	Part 2
3	KsponSpeech 의 이중 전사 · 잡음 태그 · PCM 은 어떻게 다루나 ?	Part 3
4	형식이 제각각인 데이터를 어떻게 한 장의 목록표로 모으나 ?	Part 4 ~ 5
5	누수 없이 나누고 Whisper 가 먹는 모양으로 어떻게 바꾸나 ?	Part 6

▶ 애니메이션 D11-1 오늘의 흐름

세 가지 형식 → 목록표 한 장 → Whisper 입력

D11 · 3주차 시작: 우리 데이터로 Whisper를 고쳐 쓰려면, 먼저 '깨끗한 데이터'부터



D11 데이터 준비 → D12 Full Fine-tuning → D13 LoRA → D14 비교 · 상용 API → D15 미니 프로젝트

교시	내용	실습
1 교시	데이터셋 지도 , HF datasets 스트리밍 · 바이트로 소리 읽기	Part 0 ~ 2 · 빈칸 1 ~ 2
2 교시	KsponSpeech 전사 규칙 정제 , 머리말 없는 PCM 읽기	Part 3 · 빈칸 3 ~ 5
3 교시	AIHub 소리 + JSON → 목록표 , 16 kHz 변환 · 품질 필터	Part 4 ~ 5 · 빈칸 6 ~ 8
4 교시	화자 단위 분할 , Whisper input_features · labels, 저장	Part 6 ~ 7 · 빈칸 9 · 노션

Fine-tuning 의 성패는 데이터가 정한다

🍎 요리 재료 손질

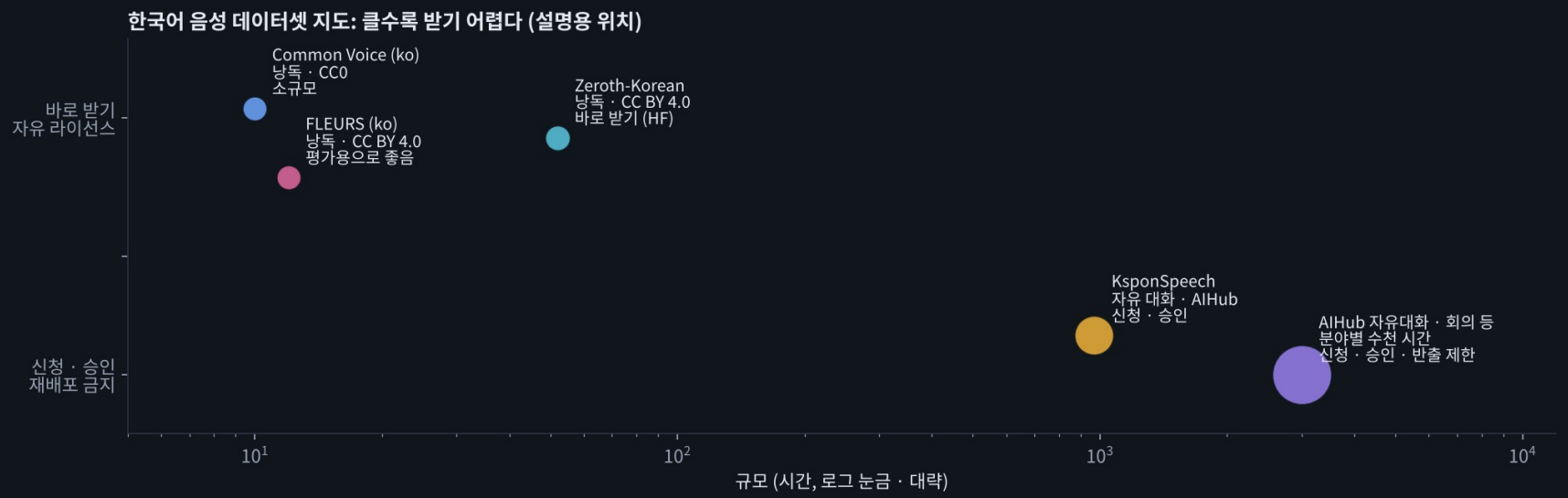
좋은 요리사 (Whisper) 라도 상한 재료 (틀린 전사 · 잘린 소리) 를 넣으면 맛이 망가진다 . 오늘은 장보기 (데이터셋 고르기) 와 손질 (정제 · 필터 · 분할) 을 하는 날이고 , 요리 (학습) 는 내일이다 .

1. 서비스와 비슷한 데이터 고르기
2. 형식 달라도 목록표 한 장으로
3. 들어 보고 기준 정해 거르기
4. 화자 단위로 나눠 공정하게 평가
5. Whisper 입력으로 바꿔 저장

💬 **쉽게 말하면** 모델을 바꾸는 것보다 데이터 100 시간을 깨끗하게 만드는 것이 더 효과가 클 때가 많다 .

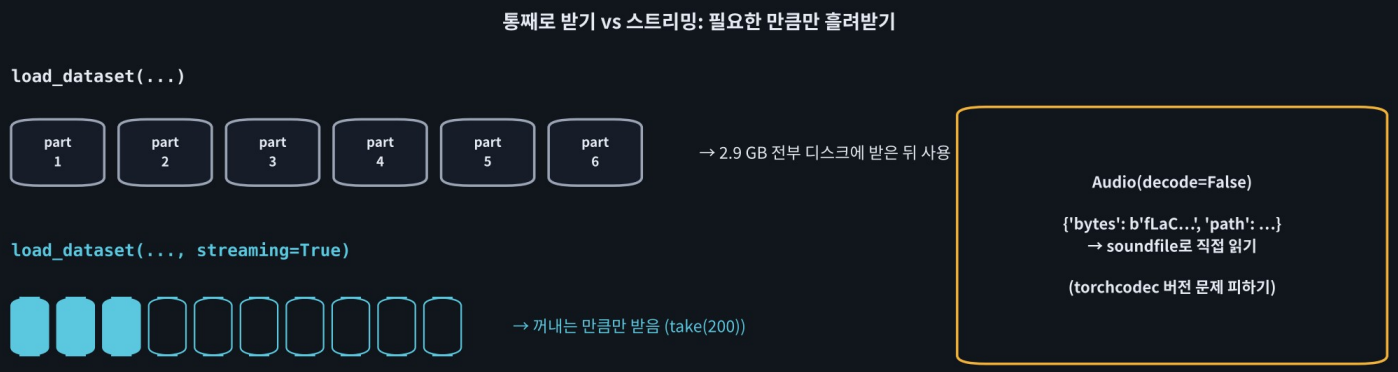
▶ 애니메이션 D11-2 데이터셋 지도

클수록 받기 어렵다



데이터셋	말투	규모	받는 법 · 라이선스
Zeroth-Korean	뉴스 문장 낭독	학습 51.6h · 105 명 / 시험 1.2h · 10 명	Hugging Face kresnik/zeroth_korean · CC BY 4.0
KsponSpeech	2 인 자유 대화	약 1,000h · 약 2,000 명	AIHub 신청 · 승인 · 재배포 금지
AIHub 분야별	회의 · 콜센터 · 명령어 · 노인 …	데이터셋마다 수백 ~ 수천 h	AIHub 신청 · 이용 약관 · 반출 제한
Common Voice · FLEURS	문장 낭독	수 ~ 수십 h	CC0 · CC BY 4.0 (평가용으로 좋음)

필요한 만큼만 흘려받기



🍎 영화 다운로드 vs 스트리밍

앞부분만 보고 싶으면 전체 파일을 받을 필요 없이 스트리밍으로 바로 본다 . 여러 번 볼 영화라면 한 번 받아 두는 게 낫다 .

▶ 애니메이션 D11-3 다운로드 vs 스트리밍

```
from datasets import load_dataset, Audio as HFAudio
stream = load_dataset("kresnik/zeroth_korean", split="train",
                      streaming=True)
stream = stream.cast_column("audio", HFAudio(decode=False))
for ex in stream.take(3): print(ex["text"])
```

🧩 빈칸 실습 1

streaming=___ → True

ex["audio"]["bytes"] → sf.read

```
import io, soundfile as sf
ex = next(iter(stream))
y, sr = sf.read(io.BytesIO(ex["audio"]["bytes"]))
print(sr, len(y) / sr, ex["text"], ex["speaker_id"])
# 16000  4.1  선생님께서 …  105
```

열	내용
audio	{"bytes", "path"}
text	전사
speaker_id	화자 번호
chapter_id · id	묶음 · 발화 ID

🧩 **빈칸 실습 2**

ex["audio"][____] → "bytes"

💬 **쉽게 말하면** BytesIO 는 메모리 속 바이트를 " 파일인 척 " 보이게 한다 . 그래서 파일 경로를 받는 sf.read 에 그대로 넣을 수 있다 .

받은 데이터부터 재 본다

```
zt = load_dataset("kresnik/zeroth_korean", split="test")
zt = zt.cast_column("audio", HFAudio(decode=False))
for ex in zt:
    y, sr = sf.read(io.BytesIO(ex["audio"]["bytes"]))
    sf.write(f"zeroth_test/{ex['id']}.flac", y, sr)
# 457 개 · 약 1.2 시간 · 화자 10 명
```

오늘은 수업 시간 때문에 작은 시험 세트 (약 60 MB) 를 연습 재료로 씁니다 . 실제 Fine-tuning 에서는 학습 세트로 학습하고 공식 시험 세트는 마지막 평가에만 씁니다 .

들은 그대로 적고 , 기호로 표시한다

KsponSpeech 전사 규칙: 들은 그대로 적고, 기호로 표시한다

원본 o/ 그럼 (3시)/(세 시)에 b/ 회의+ 회의를 할까/ 요* 음/

(A)/(B)	이중 전사: (철자)/(발음)	→ 하나 고르기
b/ l/ o/ n/ u/	잡음 · 웃음 · 겹침 · 기타	→ 지우기
+	반복 · 더듬음	→ 기호 지우기
*	알아듣기 어려움	→ 기호 지우기 (또는 문장 버리기)
단어/	간투어 (음/, 어/)	→ 기호 지우기 또는 단어 삭제

철자 모드
그럼 3시에 회의 회의를 할까 요 음

발음 모드
그럼 세 시에 회의 회의를 할까 요 음

기호	뜻	정리
(3 시)/(세 시)	이중 전사 : 철자 / 발음	한쪽 고르기 (학습 · 평가 같은 규칙)
b/ l/ o/ n/ u/	숨 · 웃음 · 겹침 · 잡음 · 알 수 없음	지우기
+ *	반복 · 더듬기 / 알아듣기 어려움	기호 떼기 (또는 * 문장 버리기)
음 / 어 /	간투사	/ 떼기

순서가 중요한 네 단계

```
def clean_kspon(text, mode="spelling"):
    s = re.sub(r"\(([^)]*)\)/\(([^)]*)\)",
        lambda m: m.group(2 if mode == "phonetic" else 1), text)
    s = re.sub(r"[a-z]/", "", s)          # 잡음 태그
    s = s.replace("+", "").replace("*", "").replace("/", "")
    return re.sub(r"\s+", " ", s).strip()
```

```
o/ 그럼 (3 시 )/( 세 시 ) 에 b/
회의 + 회의를 할까 / 요 * 음 /
→ 그럼 3 시에 회의 회의를
  할까 요 음
```

▶ 애니메이션 D11-4 전사 정제기

🧩 빈칸 실습 3

else ____ → 1 (철자)

🧩 빈칸 실습 4

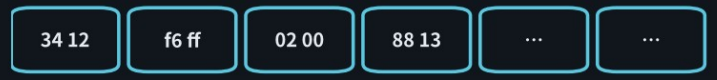
.replace(____, "") → "*"

.pcm = 소리 숫자만 줄줄이

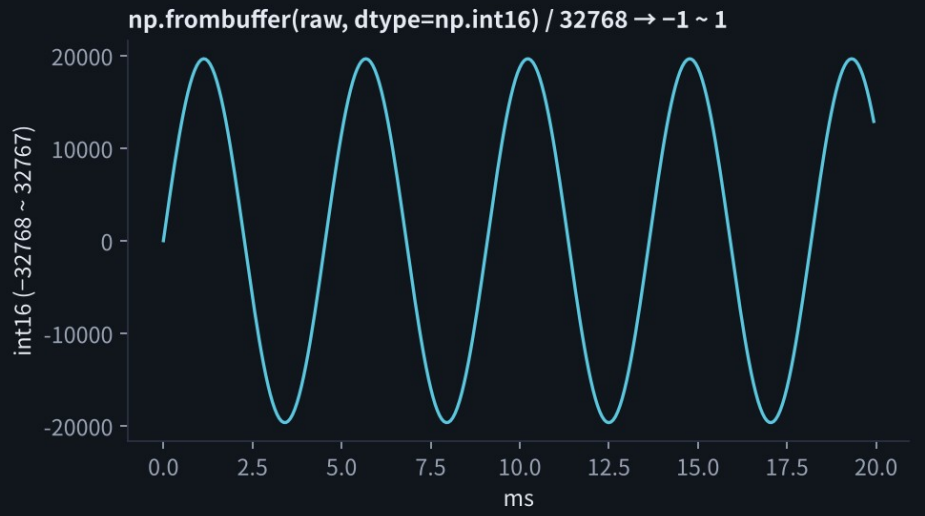
.wav = 머리말(44바이트) + 소리 숫자



.pcm (KsponSpeech) = 머리말 없이 소리 숫자만



2바이트(16bit) = 숫자 하나, 작은 쪽 먼저(little-endian): 34 12 → 0x1234 = 4660




```
raw = open(path, "rb").read()
y = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768
```

▶ 애니메이션 D11-5 PCM 바이트 읽기

64,000 바이트 ÷ 2 = 32,000 샘플 ÷ 16,000 = 2 초

🧩 **빈칸 실습 5**

dtype=____ → np.int16 (2 바이트 = 샘플 하나)

원천데이터 (소리) + 라벨링데이터 (JSON)

```
aihub_mini/
 ├── 원천데이터 /S001/S001_0001.wav
 └── 라벨링데이터 /S001/S001_0001.json

{" 발화정보 ": {"fileNm": "S001_0001.wav",
  "stt": " 오늘 회의는 … ", "recrdTime": 3.21},
 " 녹음자정보 ": {"recorderId": "S001", …}}
```

```
files = sorted(glob("aihub_mini/ 라벨링데이터 /**/*.json",
                    recursive=True))
def parse_label(jp):
    lab = json.load(open(jp, encoding="utf-8"))
    wav = jp.replace(" 라벨링데이터 ", " 원천데이터 ")
            .replace(".json", ".wav")
    info = sf.info(wav)  # 길이 · 주파수 · 채널
```

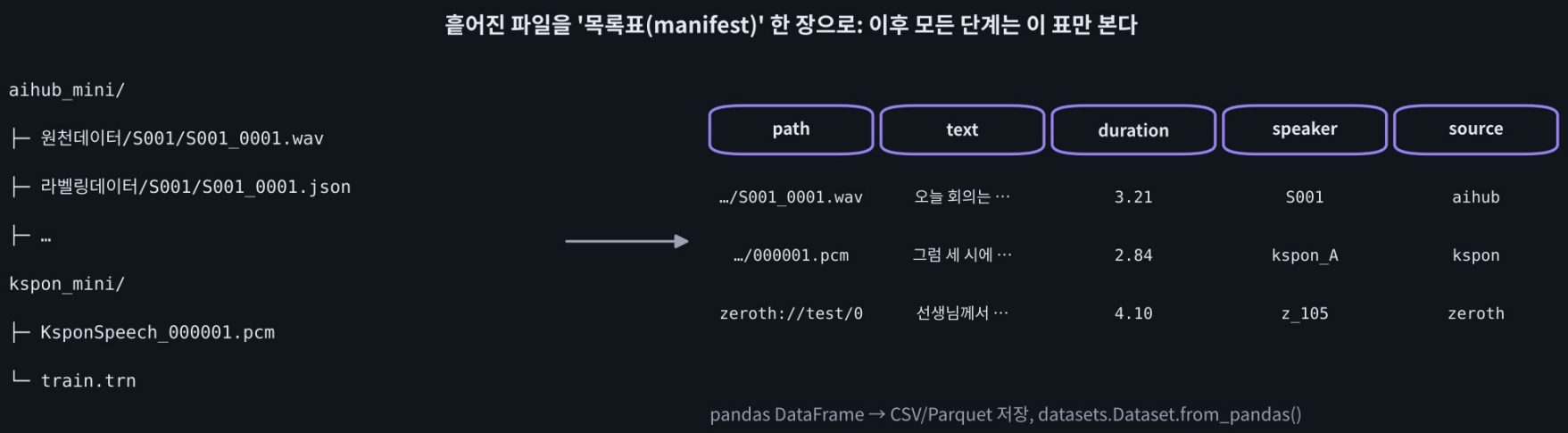
🧩 **빈칸 실습 6**

recursive=____ → True (** = 하위 폴더 전부)

💬 **쉽게 말하면** " 짝 찾기 ": 라벨 경로의 폴더 이름과 확장자만 바꾸면 소리 경로가 된다 .

▶ 애니메이션 D11-6 목록표 만들기

흩어진 파일을 표 한 장으로



열	쓰임
path · text	학습 입력 · 정답
duration	길이 필터 · CPS · 전체 시간 계산
speaker	화자 단위 분할
source	출처별 통계 · 라이선스 관리

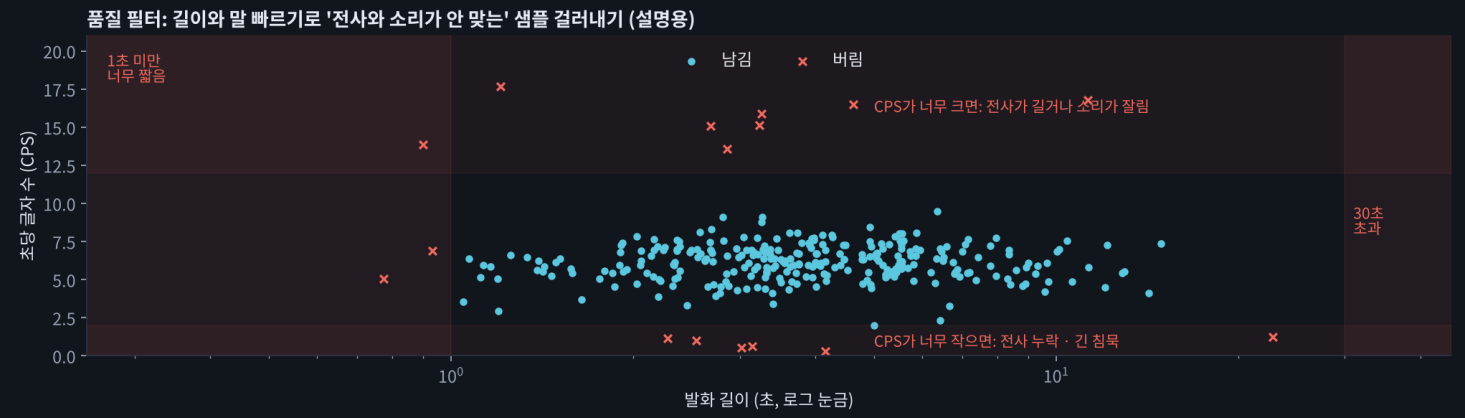
모든 소리 → 16 kHz 모노 float

```
def load_16k(path):
    if path.endswith(".pcm"): return read_pcm(path)
    y, sr = sf.read(path, dtype="float32", always_2d=True)
    y = y.mean(axis=1)                  #  스테레오 → 모노
    if sr != 16000:
        y = librosa.resample(y, orig_sr=sr, target_sr=16000)
    return y
```

출처	원래 형식
zeroth	flac 16 kHz 모노
kspon	pcm 16 kHz (머리말 없음)
aihub	wav 16 / 44.1 kHz, 모노 · 스테레오

💬 **쉽게 말하면** 규격이 제각각인 콘센트에 어댑터 하나를 끼우는 것 : 뒤 단계는 "16 kHz 모노 " 만 생각하면 된다 .

길이와 말 빠르기로 " 안 맞는 " 샘플 거르기



심어 둔 문제	걸리는 규칙
빈 전사	text == ""
9 초 침묵 꼬리	CPS < 2
긴 소리 + " 네 ."	CPS < 2
30 초 초과	duration > 30

▶ 애니메이션 D11-7 품질 필터

```
df_all["cps"] = df_all.n_chars / df_all["duration"]
reason[(reason == "") & (df_all.duration > 30)] = " 너무 김 "
#  빈 전사 · 1 초 미만 · CPS > 12 · CPS < 2  도 같은 방식
```

🧩 빈칸 실습 7

duration > ____ → 30

🧩 빈칸 실습 8

df_all[____] → "duration"

시험 화자는 학습에 없어야 한다



색 = 화자. 시험 점수가 '처음 듣는 목소리'에 대한 실력이 되려면 화자 단위로 나눈다 (GroupShuffleSplit)

```
gss = GroupShuffleSplit(n_splits=1, test_size=test_size, random_state=seed)
a, b = next(gss.split(df, groups=df[group]))
GROUP = "speaker"; rest, test = split_by(df_ok, 0.2, group=GROUP)
```

▶ 애니메이션 D11-8 데이터 누수 실험

assert: set(train.speaker) & set(test.speaker) == 빈 집합

🧩 **빈칸 실습 9**

GROUP = ____ → "speaker"

train · valid · test

묶음	쓰임	언제 보나
train	모델이 배우는 데이터	매 학습 단계
valid	하이퍼파라미터 · 체크포인트 선택	학습 중 주기적으로
test	최종 성적	맨 마지막 한 번

🍎 **연습문제 · 모의고사 · 수능**

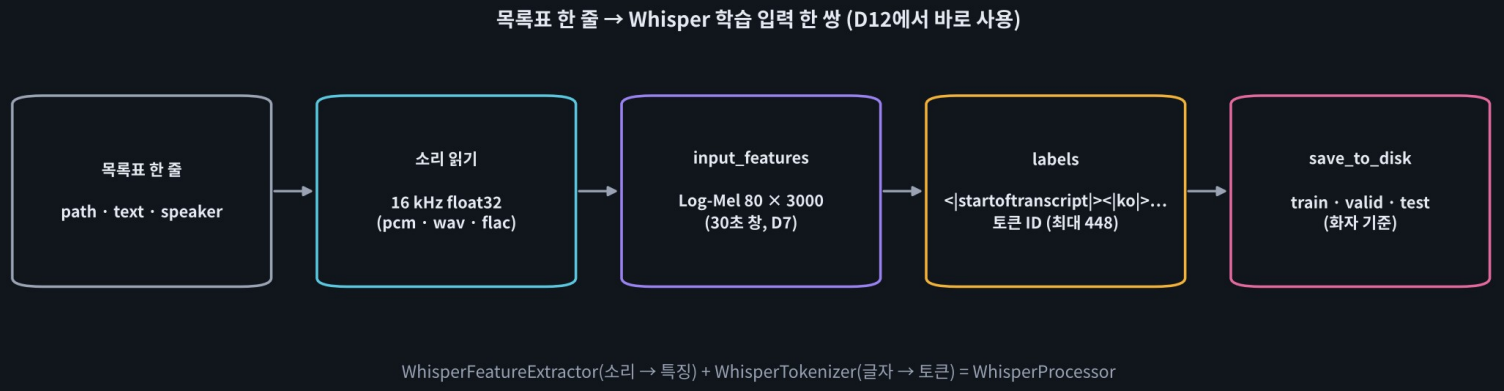
연습문제 (train) 로 공부하고 모의고사 (valid) 로 공부법을 고르고 , 수능 (test) 은 한 번만 본다 . 수능 문제를 미리 보면 점수는 의미가 없다 .

💬 **쉽게 말하면** test 를 보고 설정을 바꾸기 시작하면 test 도 사실상 valid 가 된다 .

```
rest, test = split_by(df_ok, 0.2, group=GROUP)     # 화자 기준 test 20%
train, valid = split_by(rest, 0.15, group=GROUP)   # 남은 것에서 valid 15%
assert not overlap   # 세 묶음 사이 겹치는 화자 없음
# 출처별 개수도 함께 확인 : 한 묶음에 한 출처만 몰리지 않았는지
```

▶ 애니메이션 D11-9 Whisper 입력

목록표 한 줄 → input_features + labels



WhisperFeatureExtractor(소리 → 특징) + WhisperTokenizer(글자 → 토큰) = WhisperProcessor

```
processor = WhisperProcessor.from_pretrained("openai/whisper-small",
                language="korean", task="transcribe")
def prepare(row):
    y = load_16k(row["path"])
    row["input_features"] = processor.feature_extractor(
        y, sampling_rate=SR).input_features[0]
    row["labels"] = processor.tokenizer(row["text"]).input_ids
    return row
```

항목	크기
input_features	80 × 3000
labels	최대 448
소리 길이	≤ 30 초

save_to_disk → 내일 바로 불러오기

```
dsd = DatasetDict({n: Dataset.from_pandas(d[cols]) for n, d in ...})
dsd = dsd.map(prepare, desc="Whisper 입력 만들기")
dsd.save_to_disk("ko_asr_prepared")
...to_csv("ko_asr_manifest.csv", index=False)
# D12: load_from_disk("ko_asr_prepared")
```

방식	디스크	계산
map + 저장	많이 씀	한 번
with_transform	적게 씀	매번

Colab 세션이 끝나면 파일이 사라집니다. Google Drive 에 복사해 두세요.

1. D12: load_from_disk("ko_asr_prepared") 로 불러오기
2. 데이터 콜레이터로 배치마다 labels 패딩 (-100)
3. Seq2SeqTrainer 로 Whisper 전체 학습 · CER 로 평가

오늘 만든 데이터로 내일 Whisper 를 고쳐 쓴다

- ☐ 한국어 음성 데이터셋의 말투 · 규모 · 라이선스를 비교했다
- ☐ HF datasets 를 스트리밍하고 바이트로 소리를 읽었다
- ☐ KsponSpeech 전사를 정제하고 PCM 을 읽었다
- ☐ AIHub 형식에서 목록표를 만들었다
- ☐ 16 kHz 변환 · 품질 필터로 문제 샘플을 걸렀다
- ☐ 화자 단위로 나누고 Whisper 입력으로 저장했다

다음 : D12 Whisper Full Fine-tuning

오늘 저장한 ko_asr_prepared 를 불러와 Seq2SeqTrainer 로 Whisper 전체를 학습합니다. 데이터 콜레이터 (패딩) · 학습률 · 평가 지표 (D10 CER) 를 연결하고, 학습 전 · 후 CER 을 비교합니다.

worksheet.html	40 문항
개념 워크시트 · 노션 제출	
animation.html	9 개 탭
인터랙티브 애니메이션	
D11_한국어음성데이터셋_실습.ipynb	빈칸 9
실습 노트북	

온라인 자료 imguru-mooc.github.io/AI_Voice/day11